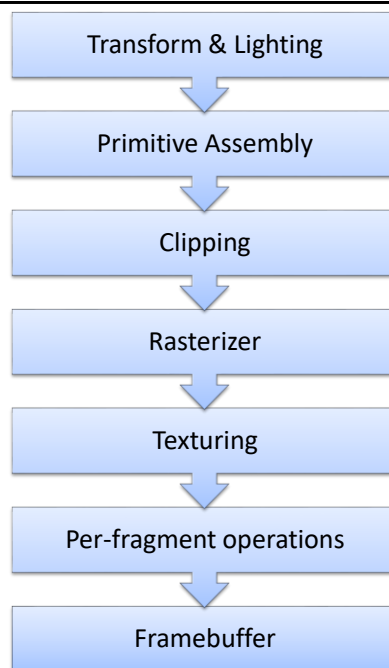


SHADER

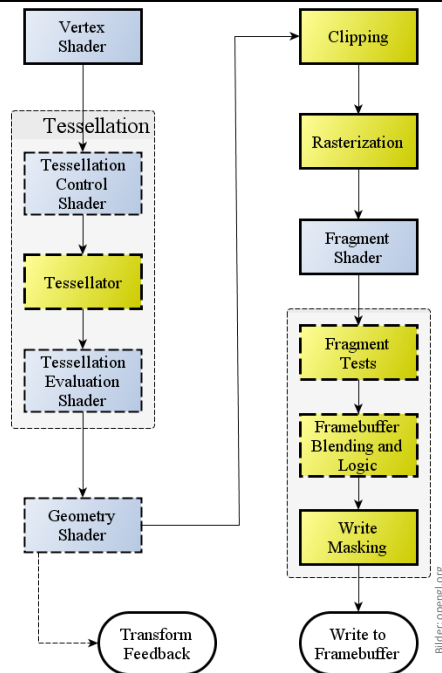
Pipeline früher

- Feste Reihenfolge
- Feste Funktionen
 - *Fixed-function pipeline*
- Kontrolle durch viele globale Variablen
 - `glEnable(...)`
 - `glShadeModel(...)`
 - ...



Pipeline heute

- Feste Reihenfolge
- Einige **Stufen** weiterhin fest
- **Andere**: Funktion frei programmierbar
 - Ein-/Ausgabe zum Teil vorgegeben
- Möglich (und notwendig) alles selbst zu programmieren



Shader

- Vertex und Fragment Shader notwendig
- Geometry und Tessellation Shader optional
- Pro Shader wird ein eigenes Programm geschrieben
 - Eigene Sprache
 - OpenGL: OpenGL Shading Language (GLSL)
 - DirectX: High Level Shading Language (HLSL)
 - Benötigt Compiler (im Grafikkarten“treiber“)

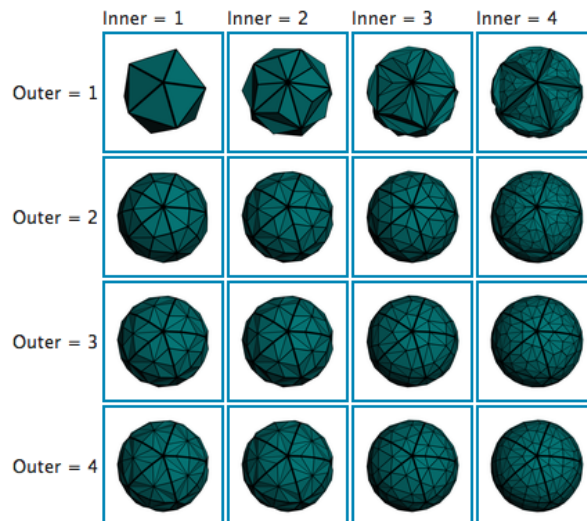
Vertex Shader

- Aufruf pro Ecke
 - Punktkoordinaten
 - Normale
 - Texturkoordinaten
 - Farbe
 - ...
- Typische Aufgaben
 - Projektion in Bildschirmraum
 - Interpolationswerte vorbereiten
 - z.B. Normale beim Phong-Shading

Tessellation Shader

- Eigentlich zwei Shader
 - Tessellation-Control-Shader
 - Tessellation-Evaluation-Shader
- Aufruf pro Fläche (Control) und pro generiertem Punkt (Evaluation)
- Typische Aufgaben
 - Unterteilung der Fläche in kleinere Flächen
 - Höhere Auflösung z.B. bei Rundungen
 - Vorbereitung für Displacement Mapping

Tessellation: Ergebnisse



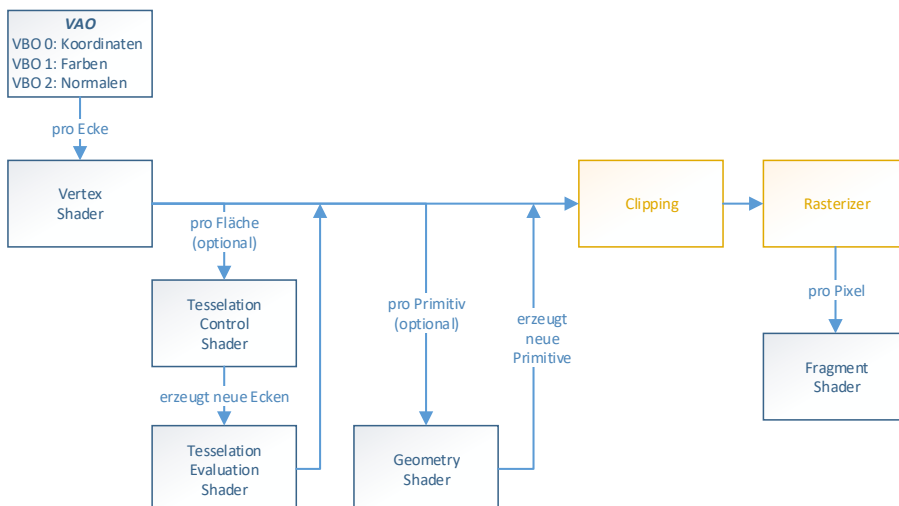
Geometry Shader

- Aufruf pro *primitive* (Punkt, Linie, Fläche)
 - Attribute der zugehörigen Ecken
 - z.B. Koordinaten
- Kann mit `EmitVertex()` neue Ecken erzeugen und mit `EndPrimitive()` neue Linien/Flächen
- Typische Aufgaben
 - Shadow volumes, cube mapping
 - Früher auch Tessellation

Fragment Shader

- Aufruf pro Fragment (=Pixel)
- Automatische Interpolation der Eingangswerte
 - z.B. der Farben oder Normalen aus den Ecken
- Typische Aufgaben
 - Farbe bestimmen
 - Phong-Shading
 - Texturen kombinieren

Noch mal Shader



Rendering-Pipeline (funktional)

1. Objekte mit lokalen Koordinaten
2. Objekte in Weltkoordinaten
3. Kamera definiert Sichtvolumen
4. Entfernen von Rückseiten
5. Zuschneiden auf Sichtvolumen
6. Rastern
7. Schattieren
8. Entfernen verdeckter Flächen

Vertex
Shader

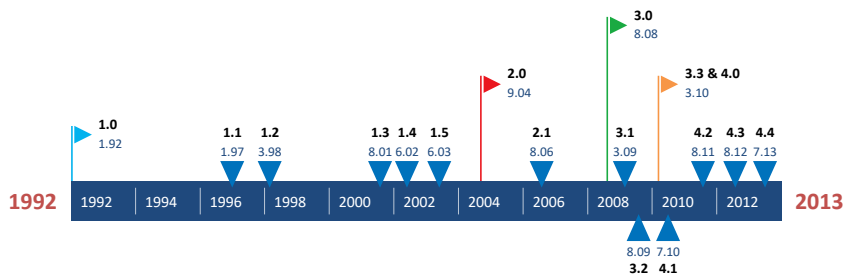
Clipping

Rasterizer

Fragment
Shader

Z-Buffer

Zeitleiste OpenGL



Shader in OpenGL

- Vertex- und Fragment-Shader gibt es seit OpenGL 2.0
 - „Fixed functions“ sind noch enthalten
- Geometry-Shader ab OpenGL 3
- Tessellation-Shader ab OpenGL 4

Shader verwenden

- Pro Shader Quellen übersetzen
 - `int shaderId = glCreateShader(GL_..._SHADER)`
 - `glShaderSource(shaderId, string)`
 - `glCompileShader(shaderId)`
- Mehrere Shader bilden ein Programm
 - `int progId = glCreateProgram()`
 - `glAttachShader(progId, shaderId)`
 - `glLinkProgram(progId)`

Shader-Programm aktivieren

- Mehrere Programme möglich, z.B. pro Fläche oder pro Objekt eines
- Immer nur eines aktiv
 - *bind*-Prinzip wie bei Texturen
- `glUseProgram(progId)`
- `glUseProgram(0)` deaktiviert aktives Programm